



Arquitetura do Sistema — PDF Document Processor

Versão: 2.0

Autor: Arquitetura de Software

Data: 2026-01-16

Status: Aprovado para MVP

1. Visão Geral

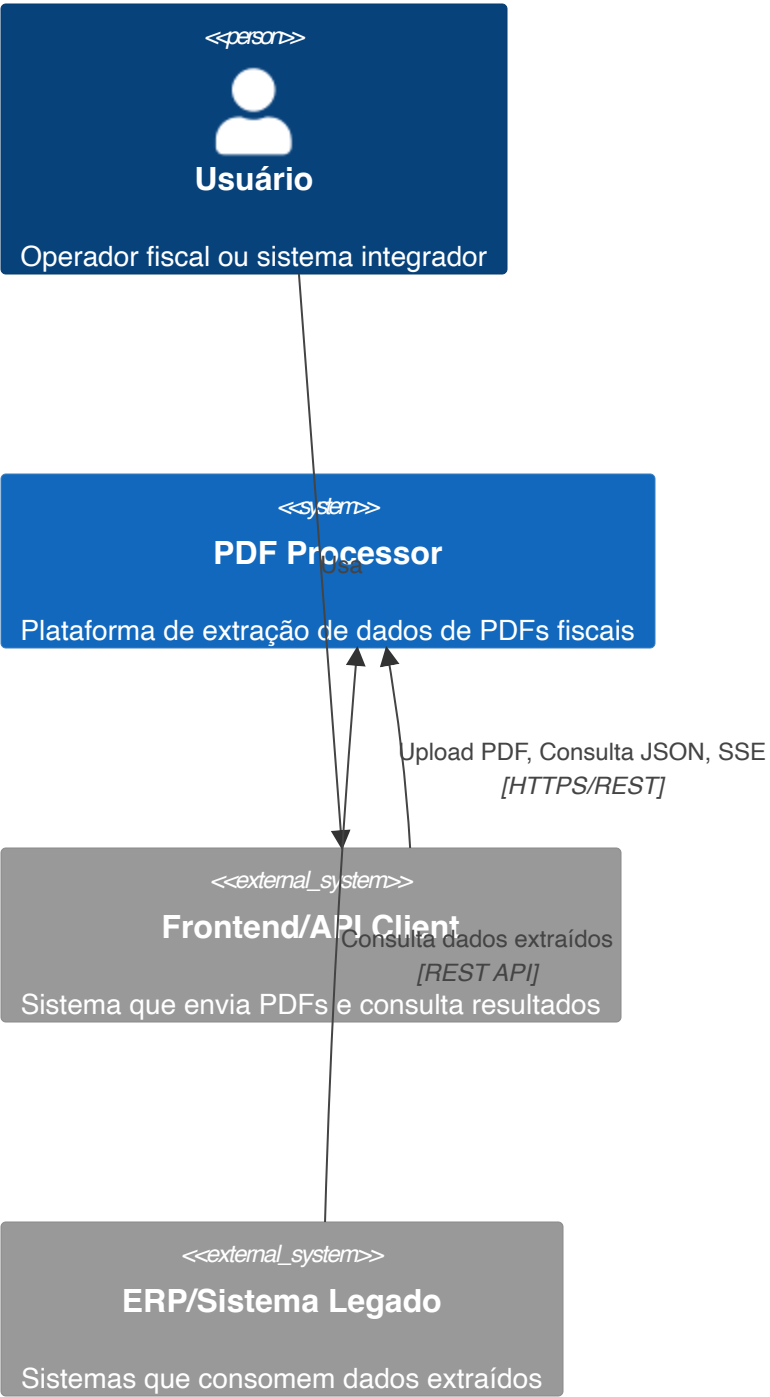
O **PDF Document Processor** é uma plataforma backend event-driven para ingestão, extração e consulta de dados de documentos fiscais em PDF (DANFES e similares). A arquitetura foi desenhada seguindo princípios de:

- **Event-Driven Architecture (EDA)** — Comunicação assíncrona via eventos
- **CQRS simplificado** — Separação entre escrita (workers) e leitura (APIs)
- **Document-Centric** — MongoDB como store principal
- **Async First** — Processamento não-bloqueante
- **Smart Extraction** — Detecção automática de PDF digital vs imagem com OCR

2. Diagrama de Contexto (C4 — Level 1)

Visão de alto nível mostrando o sistema e suas interações externas.

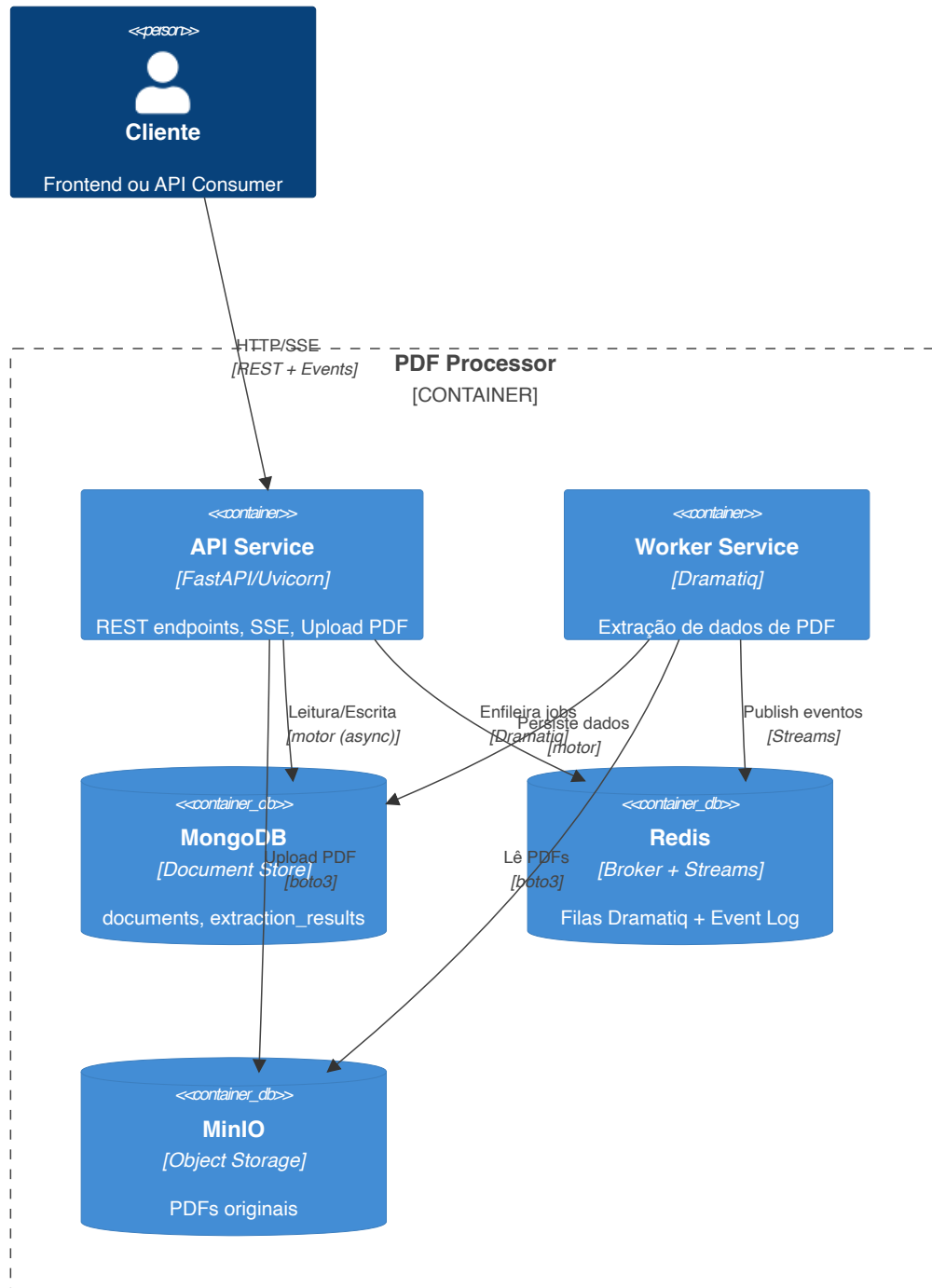
Diagrama de Contexto – PDF Document Processor



3. Diagrama de Containers (C4 — Level 2)

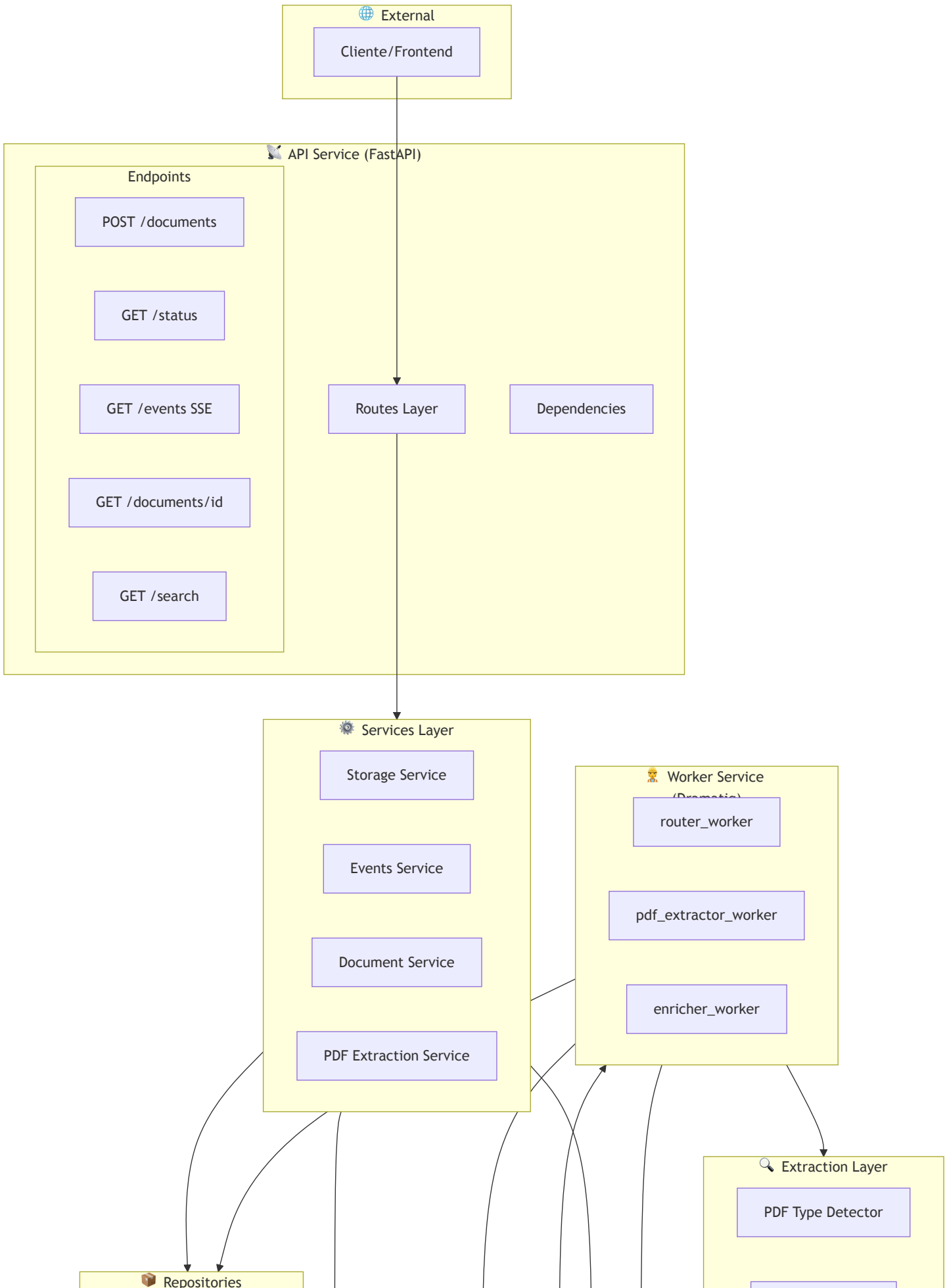
Componentes técnicos e suas responsabilidades.

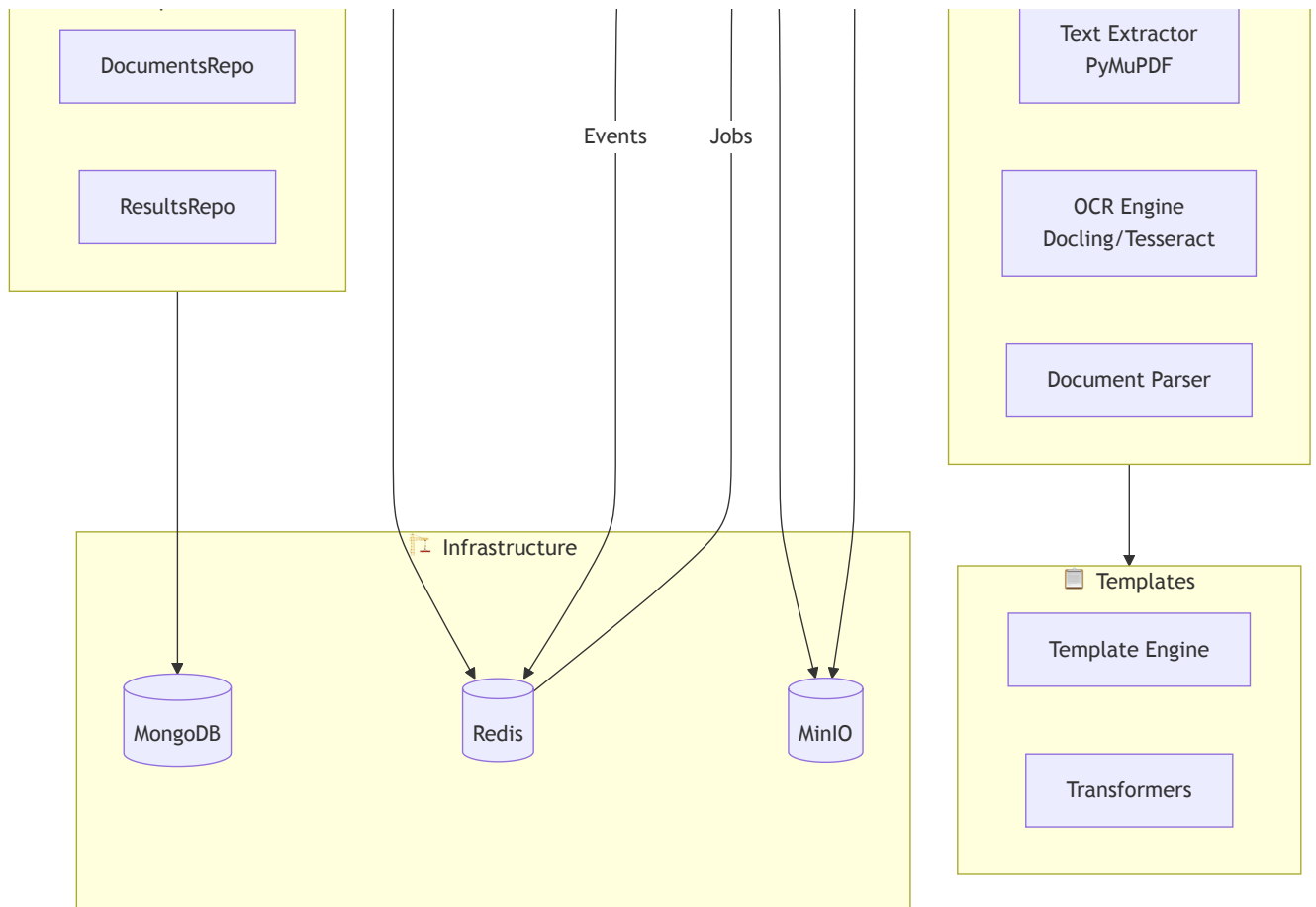
Diagrama de Containers — PDF Document Processor



4. Arquitetura de Componentes

Visão detalhada dos módulos internos.

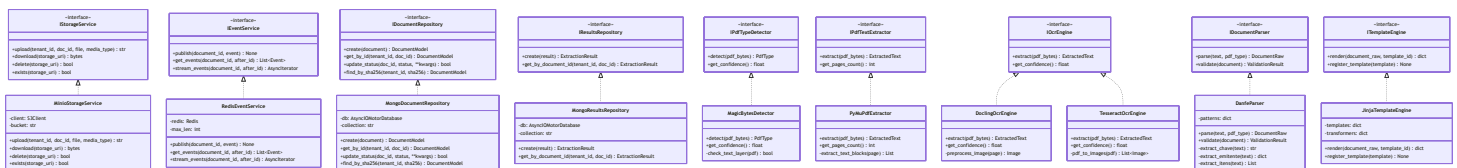




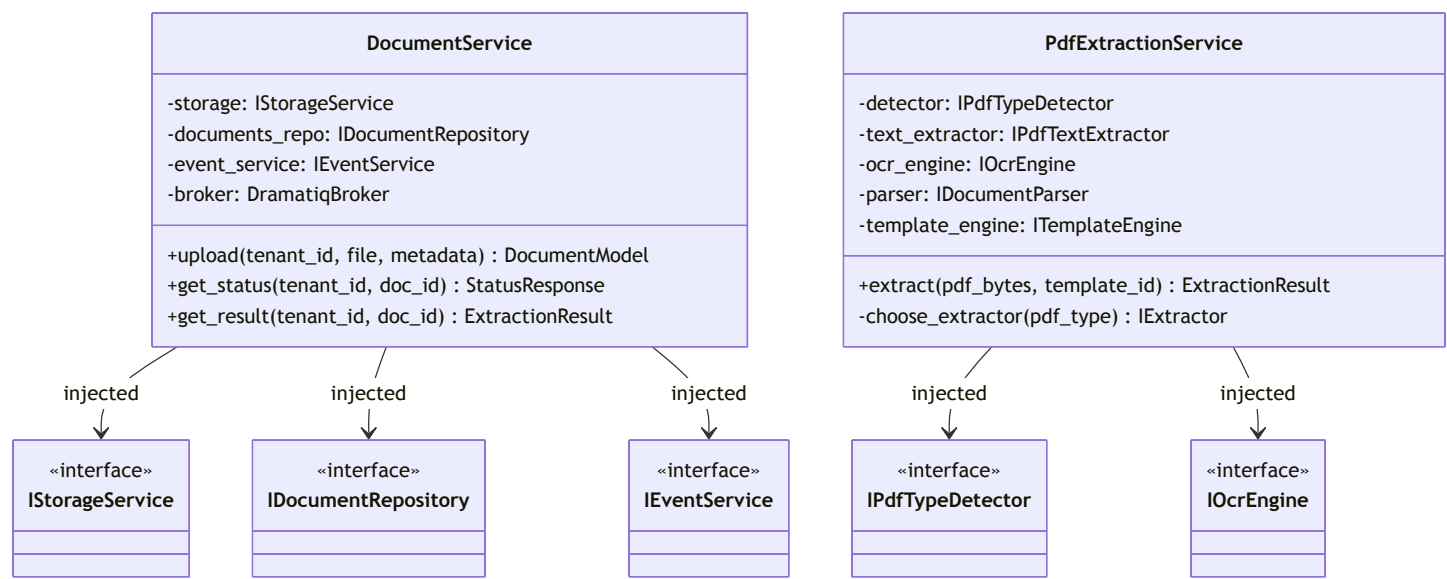
5. Diagrama de Classes e Interfaces (DI Pattern)

Estrutura de classes seguindo princípios SOLID com interfaces (Protocols) e Injeção de Dependência.

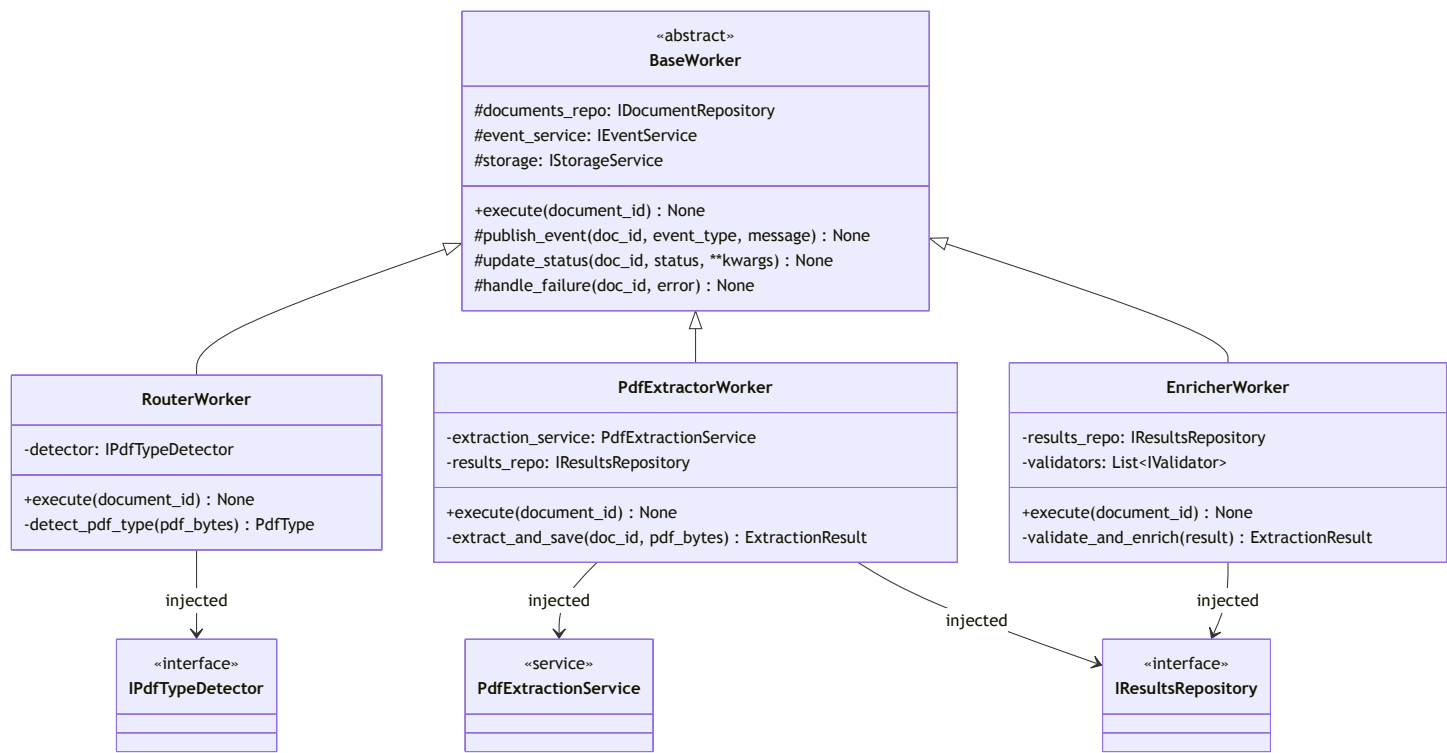
5.1 Visão Geral das Camadas



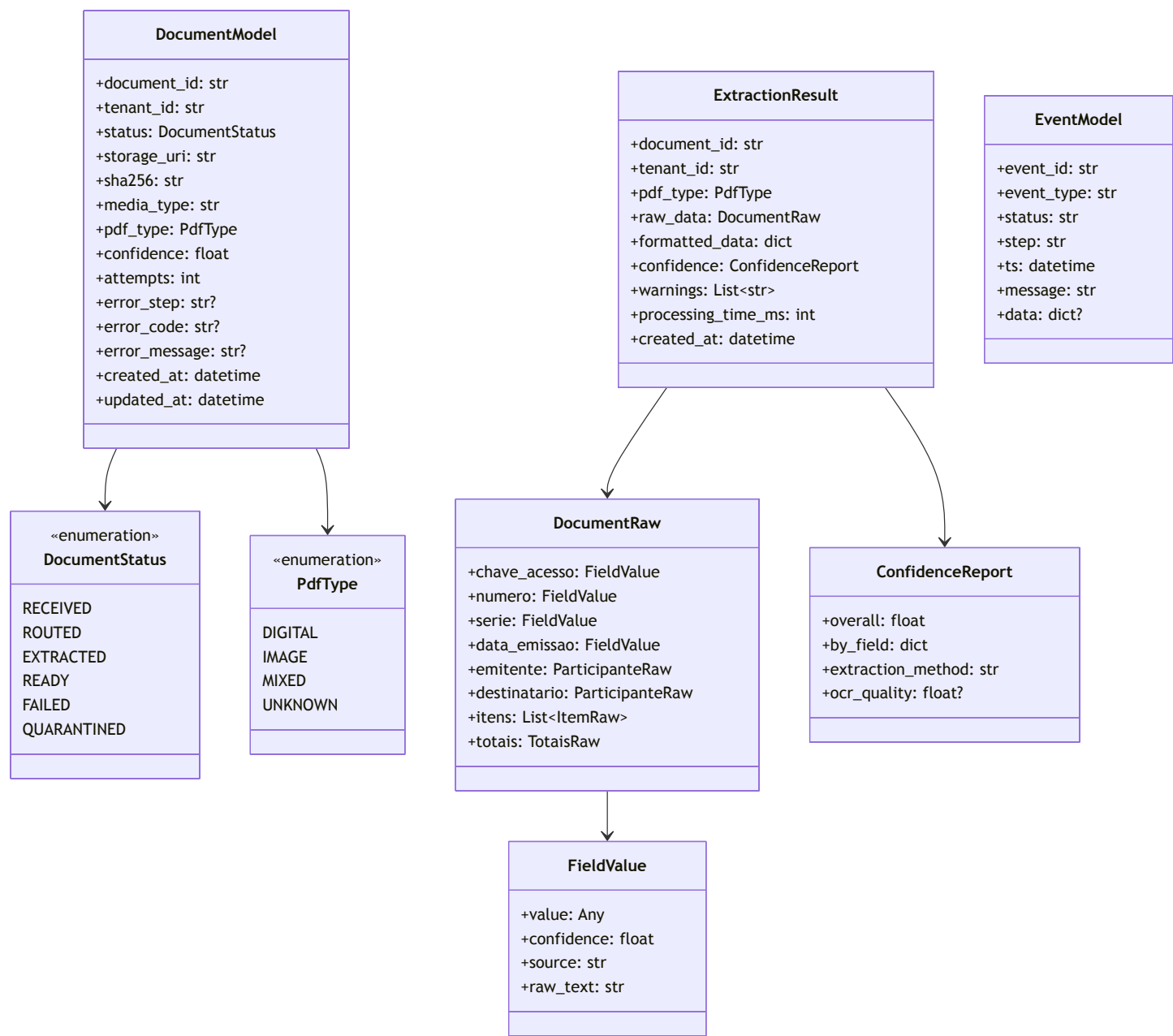
5.2 Services Layer (Business Logic)



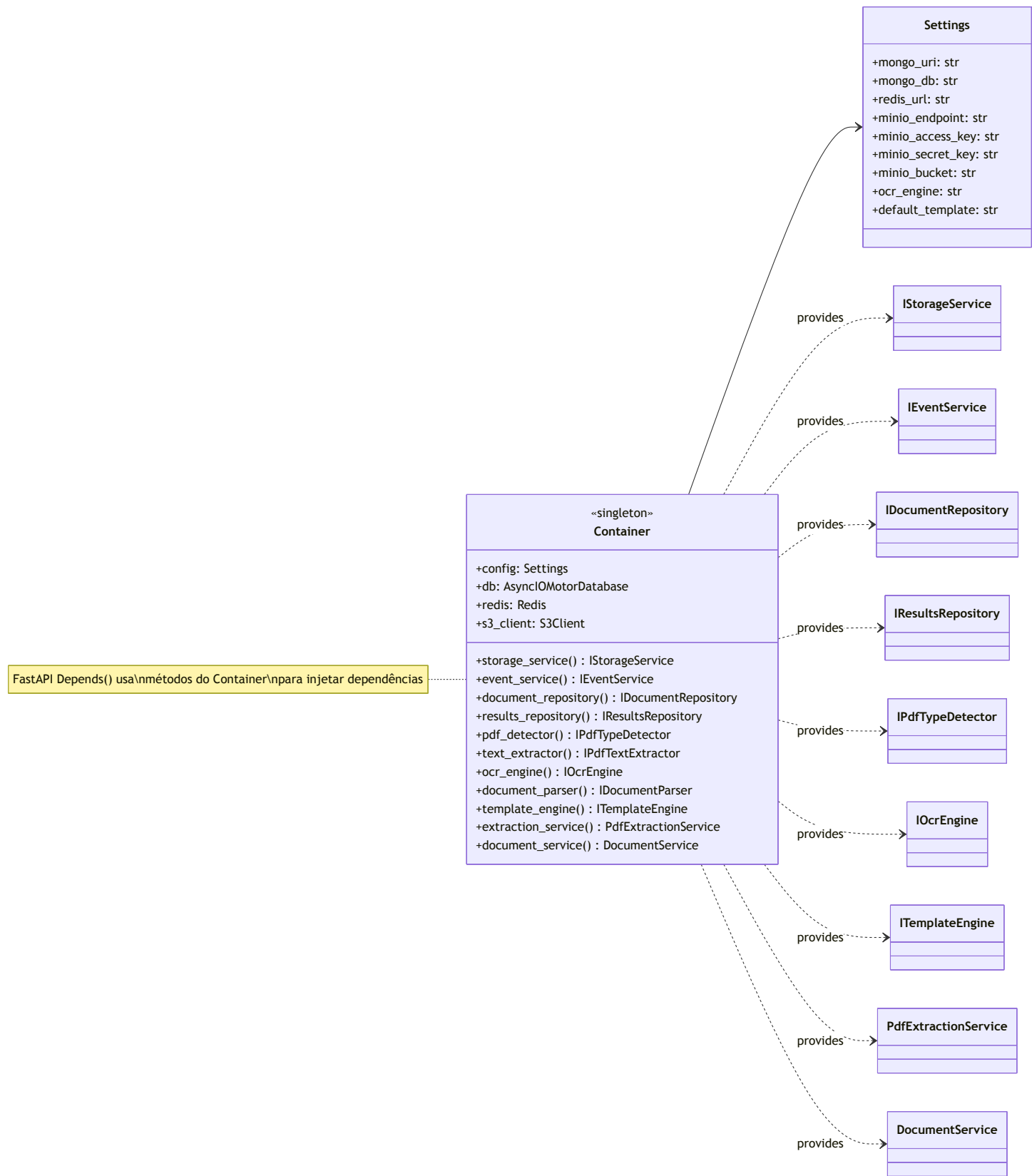
5.3 Workers (Dramatiq Actors)



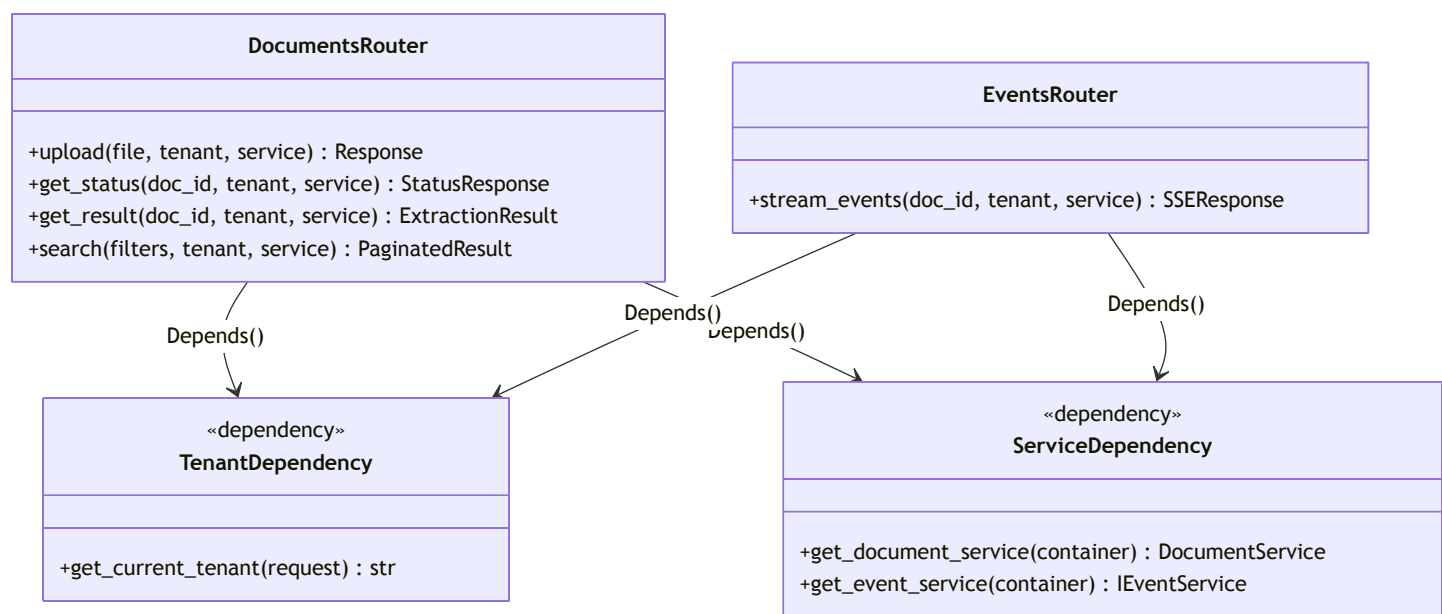
5.4 Schemas (Pydantic Models)



5.5 Dependency Injection Container



5.6 API Routes e Injeção



5.7 Princípios SOLID Aplicados

Princípio	Aplicação
S - Single Responsibility	Cada classe tem uma única responsabilidade (Detector só detecta, Extractor só extrai, Parser só parsea)
O - Open/Closed	Novos OCR engines podem ser adicionados implementando <code>I0crEngine</code> sem modificar código existente
L - Liskov Substitution	<code>Docling0crEngine</code> pode ser substituído por <code>Tesseract0crEngine</code> sem quebrar o sistema
I - Interface Segregation	Interfaces pequenas e focadas (<code>IPdfTextExtractor</code> , <code>I0crEngine</code> , <code>IDocumentParser</code>)
D - Dependency Inversion	Services dependem de abstrações (interfaces), não de implementações concretas

5.8 Exemplo de Código — DI com FastAPI

```
# container.py
class Container:
    def __init__(self, settings: Settings):
        self.settings = settings
        self._db = None
        self._redis = None

    async def storage_service(self) -> IStorageService:
        return MinioStorageService(
            endpoint=self.settings.minio_endpoint,
            access_key=self.settings.minio_access_key,
            secret_key=self.settings.minio_secret_key,
            bucket=self.settings.minio_bucket,
        )

    async def ocr_engine(self) -> IOcrEngine:
        if self.settings.ocr_engine == "docling":
            return DoclingOcrEngine()
        return TesseractOcrEngine()

    async def extraction_service(self) -> PdfExtractionService:
        return PdfExtractionService(
            detector=MagicBytesDetector(),
            text_extractor=PyMuPdfExtractor(),
            ocr_engine=await self.ocr_engine(),
            parser=DanfeParser(),
            template_engine=JinjaTemplateEngine(),
        )

# dependencies.py
async def get_document_service(
    container: Container = Depends(get_container)
) -> DocumentService:
    return await container.document_service()

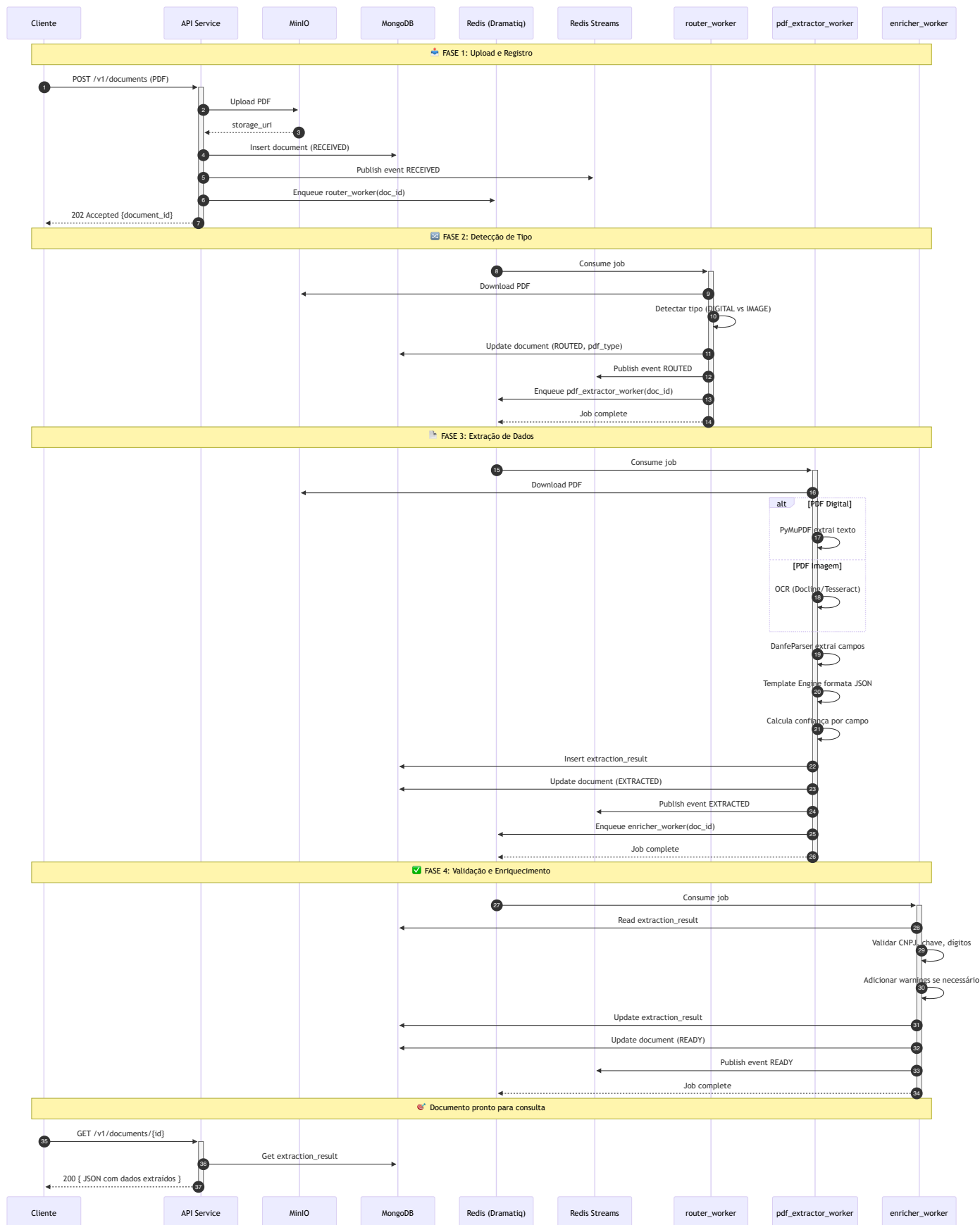
# routes/documents.py
@router.post("/documents")
async def upload_document(
    file: UploadFile,
    tenant_id: str = Depends(get_current_tenant),
    service: DocumentService = Depends(get_document_service),
```

```
) -> DocumentResponse:
    return await service.upload(tenant_id, file)

@router.get("/documents/{document_id}")
async def get_result(
    document_id: str,
    tenant_id: str = Depends(get_current_tenant),
    service: DocumentService = Depends(get_document_service),
) -> ExtractionResult:
    return await service.get_result(tenant_id, document_id)
```

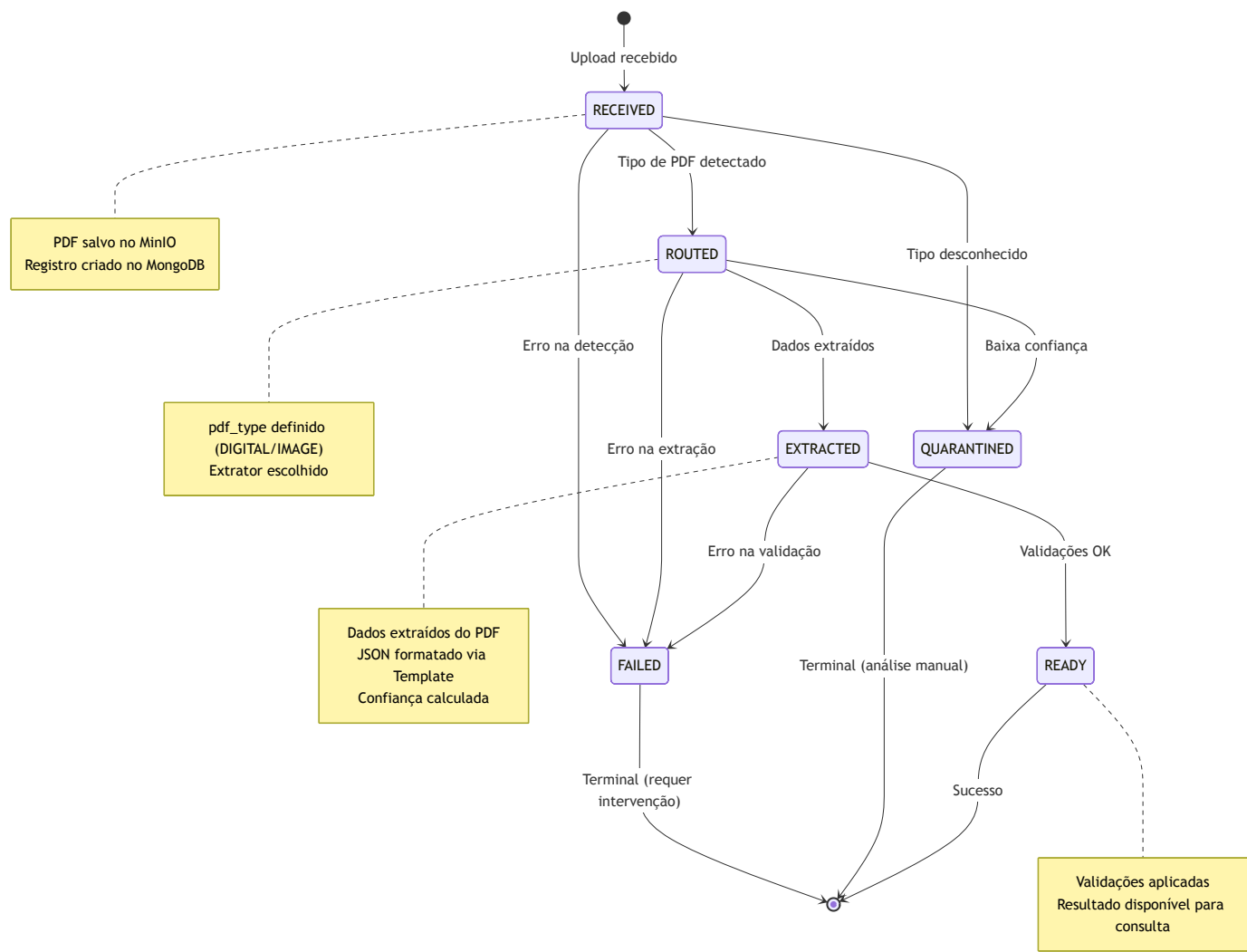
6. Fluxo de Processamento de Documento

Sequência completa desde o upload até o estado READY.



7. Máquina de Estados do Documento

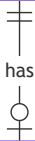
Estados e transições válidas.



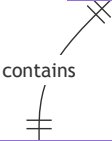
8. Modelo de Dados

Estrutura das coleções MongoDB.

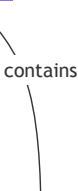
DOCUMENTS		
ObjectId	_id	PK
string	document_id	UK
string	tenant_id	FK
string	status	
string	storage_uri	
string	sha256	
string	media_type	
string	pdf_type	
float	confidence	
int	attempts	
string	error_step	
string	error_code	
string	error_message	
datetime	created_at	
datetime	updated_at	



EXTRACTION_RESULTS			
ObjectId	_id	PK	
string	document_id	FK	
string	tenant_id	FK	
string	pdf_type		
object	raw_data		Dados brutos extraídos
object	formatted_data		JSON formatado via Template
object	confidence		Confiança por campo
array	warnings		Alertas de extração
int	processing_time_ms		
datetime	created_at		



RAW_DATA

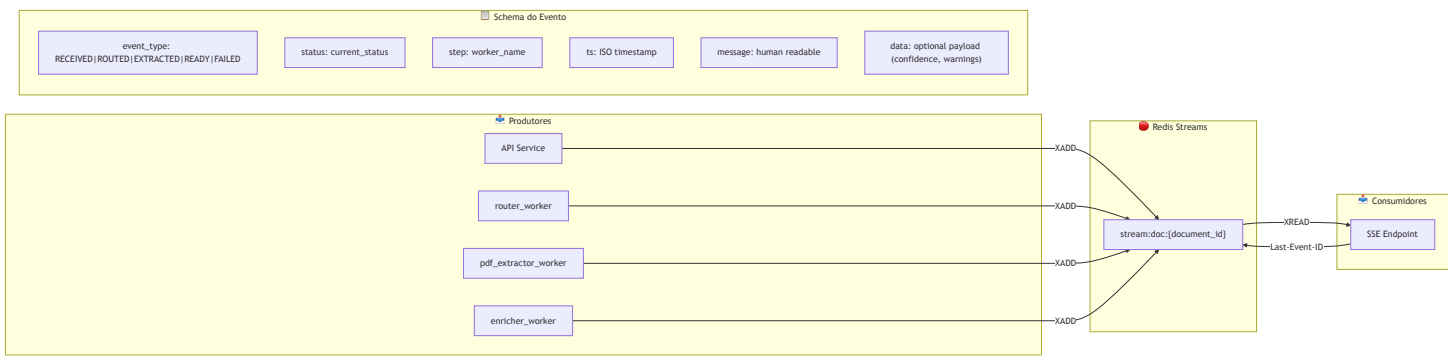


object	chave_acesso	value, confidence, source
object	numero	value, confidence, source
object	serie	value, confidence, source
object	data_emissao	value, confidence, source
object	emitente	cnpj, razao_social, etc
object	destinatario	cnpj, razao_social, etc
array	itens	Lista de produtos
object	totais	Valores totais

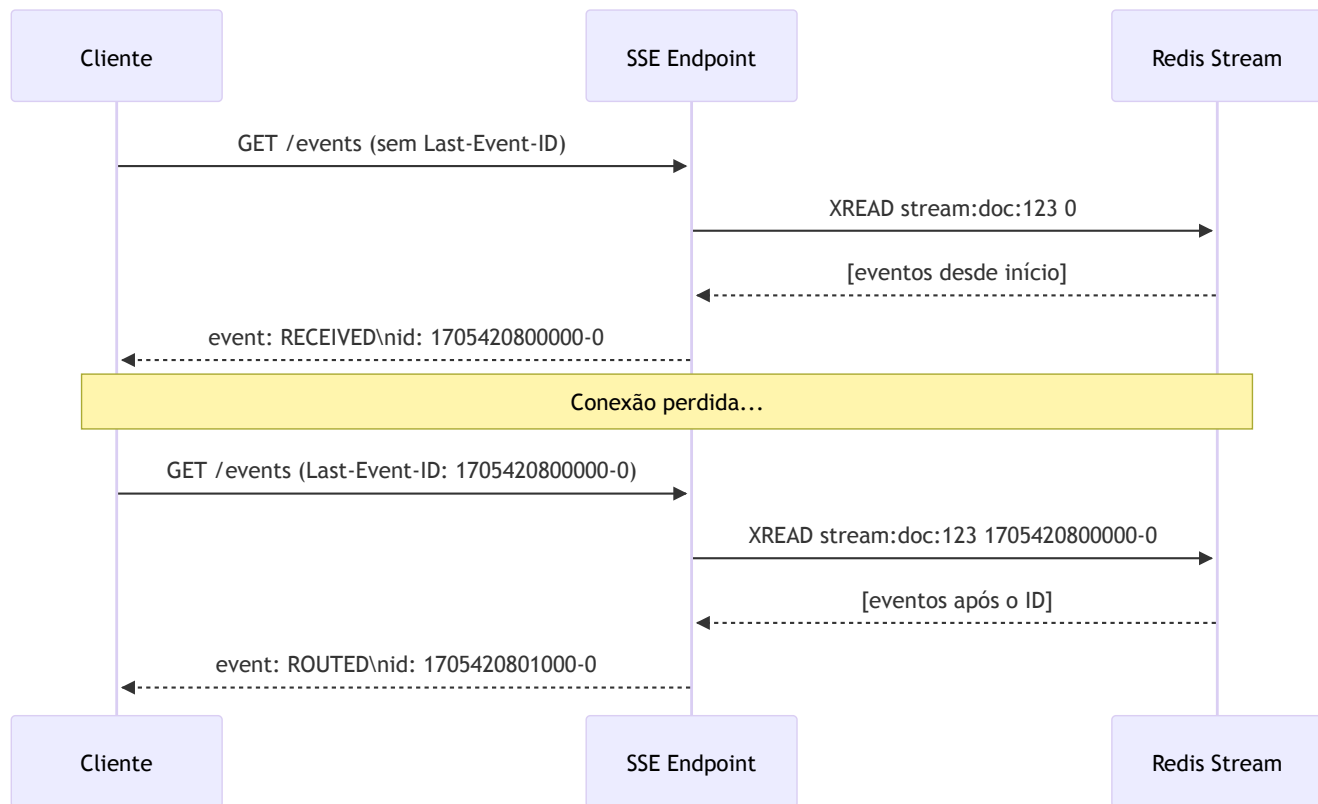
CONFIDENCE_REPORT		
float	overall	
object	by_field	Confiança por campo
string	extraction_method	DIGITAL ou OCR
float	ocr_quality	Qualidade do OCR se aplicável

9. Arquitetura de Eventos (Redis Streams)

Sistema de eventos para SSE com replay.

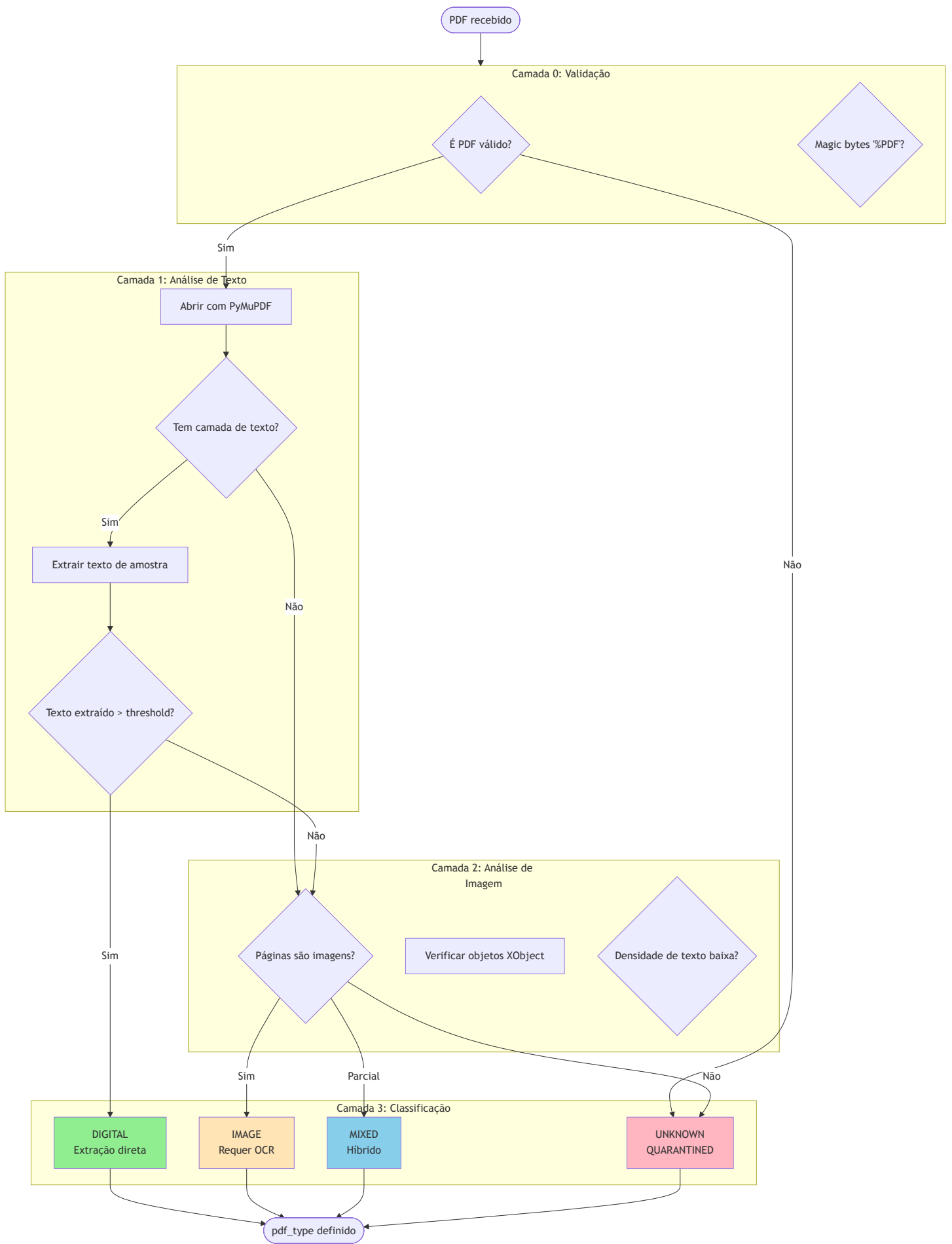


Fluxo SSE com Replay



10. Estratégia de Detecção de Tipo de PDF

Algoritmo para identificar se o PDF é digital (texto selecionável) ou imagem (escaneado).

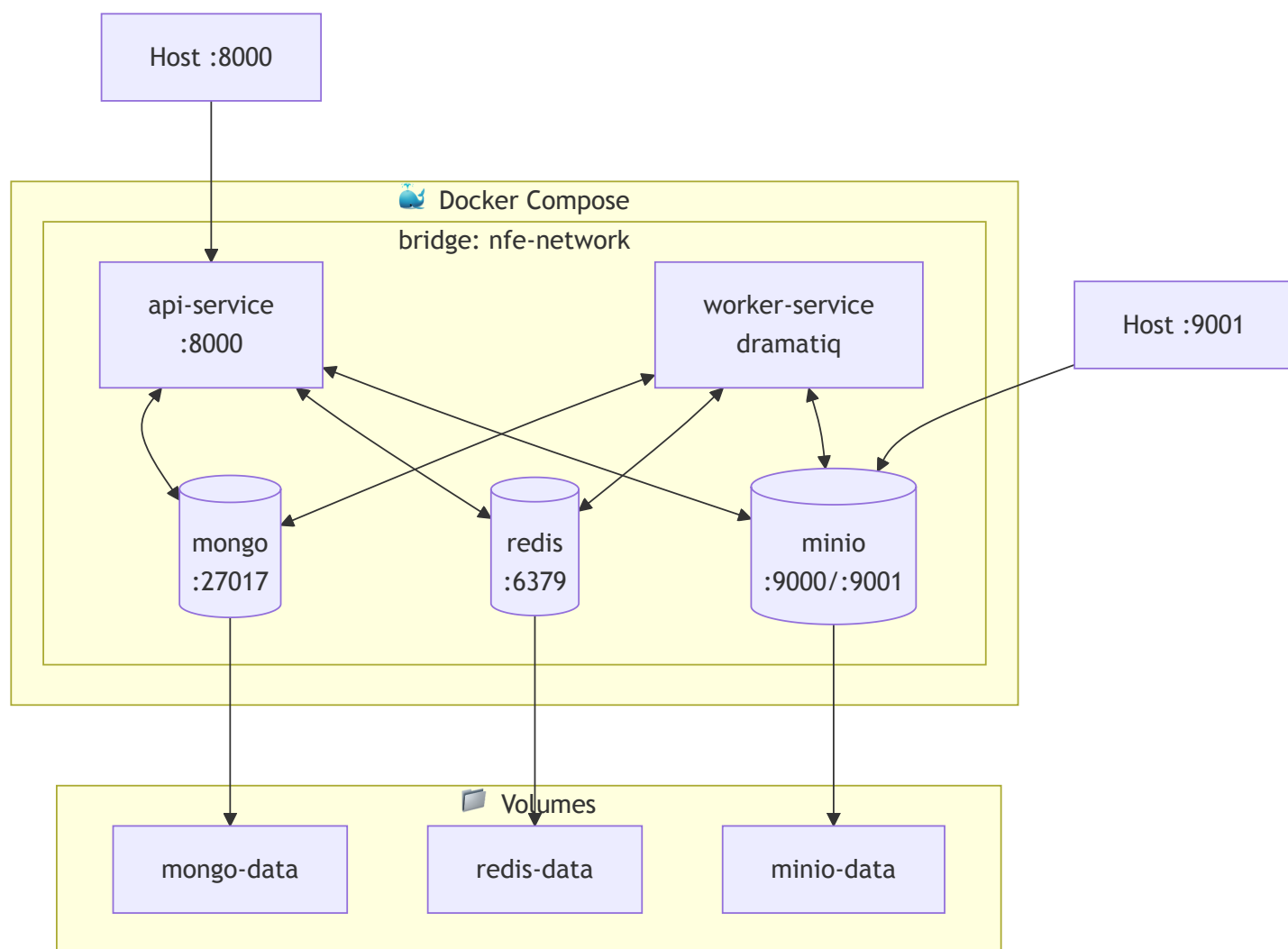


Critérios de Classificação

Tipo	Critério	Extrator
DIGITAL	Texto selecionável, > 100 chars por página	PyMuPDF/pdfplumber
IMAGE	Sem texto, páginas são imagens	Docling/Tesseract OCR
MIXED	Algumas páginas texto, outras imagem	Híbrido
UNKNOWN	PDF corrompido ou formato não suportado	Quarentena

11. Arquitetura de Deploy (Docker Compose)

Configuração de containers para ambiente local/desenvolvimento.



12. Decisões Arquiteturais (ADR Summary)

ID	Decisão	Justificativa
ADR-001	MongoDB como document store	Schema flexível para resultados, suporte a queries complexas, motor async
ADR-002	Redis Streams para eventos	Persistência de eventos, suporte a replay, baixa latência
ADR-003	Dramatiq para workers	Simples, confiável, retry built-in, Redis como broker
ADR-004	MinIO para storage	S3-compatible, self-hosted, imutabilidade de PDFs originais
ADR-005	PyMuPDF para PDF digital	Rápido, preciso, extração de texto e tabelas
ADR-006	Docling para OCR	IBM, suporte a layout, fallback para Tesseract
ADR-007	Sistema de Templates	Flexibilidade no formato de saída, compatibilidade com sistemas legados
ADR-008	Confiança por campo	Transparência na qualidade da extração, suporte a revisão manual
ADR-009	SSE com replay	Reconexão sem perda de eventos, melhor UX
ADR-010	CQRS simplificado	Workers escrevem, API lê, separação clara

13. Considerações de Segurança

🔒 Camadas de Segurança

Multi-tenant isolation
tenant_id em todas queries

Validação de entrada
Pydantic schemas

SHA256 deduplication
Evita reproprocessamento

Imutabilidade raw
Arquivos originais preservados

Audit trail
Eventos persistidos

14. Métricas e Observabilidade

Métrica	Tipo	Descrição
pdf_uploaded_total	Counter	Total de uploads
pdf_processed_total	Counter	Total processados por status
pdf_type_detected	Counter	Por tipo (DIGITAL/IMAGE)
extraction_confidence	Histogram	Distribuição de confiança
extraction_duration_seconds	Histogram	Tempo de extração
ocr_used_total	Counter	Quantas vezes OCR foi usado
worker_jobs_in_queue	Gauge	Jobs pendentes por worker
sse_connections_active	Gauge	Conexões SSE ativas

Logs Estruturados

```
{
  "timestamp": "2026-01-16T10:30:00Z",
  "level": "INFO",
  "correlation_id": "doc_abc123",
  "tenant_id": "tenant_xyz",
  "worker": "pdf_extractor_worker",
  "event": "extraction_complete",
  "pdf_type": "DIGITAL",
  "confidence": 0.95,
  "duration_ms": 1245
}
```


15. Escalabilidade Futura

MVP (Docker Compose)

1x API

1x Worker

1x Mongo

1x Redis

1x MinIO

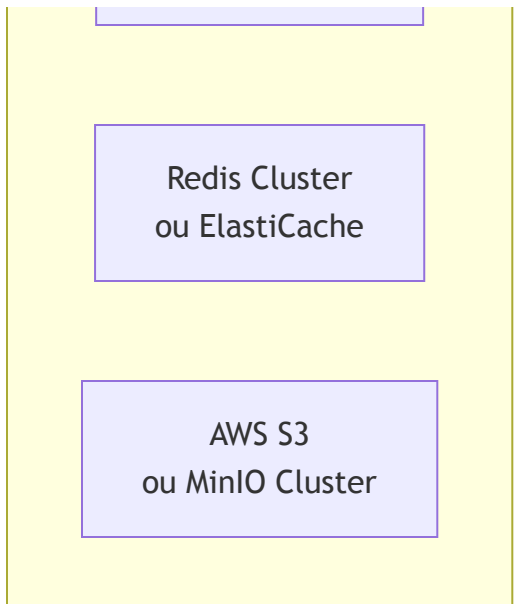
Scale

Produção (Kubernetes)

N x API pods
HPA

N x Worker pods
por tipo

MongoDB Atlas
ou ReplicaSet



16. Referências

Documentação Interna

- [Especificação Técnica](#)
- [Plano de Implementação](#)
- [Extração de PDF](#) ★ **Principal**
- [Parser e Sistema de Templates](#)

Documentação Externa

- [NFe Layout 4.00 — SEFAZ](#)
- [FastAPI Documentation](#)
- [Dramatiq Documentation](#)
- [Redis Streams](#)